

ECG Beat Graph and IBI Graph Learning

This note explains what the graph-based parts of this ECG project are trying to do, with emphasis on two modeling paths:

1. ECG beat graph learning: each node is an extracted ECG heartbeat.
2. IBI graph learning: each node is an inter-beat interval or beat-to-beat timing feature.

The short version is that the project is trying to learn a reusable ECG representation. Instead of treating ECG only as a long one-dimensional waveform, the graph models convert a recording window into a structured set of heartbeat-level objects. The model then learns not only what each beat looks like, but also how beats relate to other beats in the same window.

Overall Goal

The goal is to build an ECG foundation model that can learn from many ECG recordings and later support downstream prediction tasks. In this repository those downstream tasks appear to include infant behavioral state, sleep/status labels, and infant-versus-caregiver style classification.

The general pipeline is:

Raw ECG recording -> fixed time window -> R-peak detection -> beat or IBI nodes -> graph model -> window-level representation -> downstream prediction

The graph is useful because ECG interpretation is relational. A single beat may be noisy, normal, or ambiguous by itself. But a sequence of beats reveals rhythm, variability, repeated morphology, artifacts, and state changes. Graph learning gives the model a way to combine information across related beats instead of forcing every beat to be interpreted independently.

Why Graphs for ECG?

Traditional signal models often process ECG as a flat time series. That is useful, but it can mix several different levels of information:

- sample-level waveform shape,
- beat-level morphology,
- beat-to-beat timing,
- local rhythm,
- repeated motifs,
- noise/artifact quality,
- global state of the recording window.

A graph separates some of these levels. Each node can represent a heartbeat or an interval. Edges can represent temporal adjacency, morphological similarity, or learned attention between beats. Message passing then lets each node update its representation using its neighbors.

In this project, the graph is not a social network or anatomical graph. It is a computational structure over ECG events. The question is: which beats should exchange information with which other beats?

Part 1: ECG Beat Graph

What Is a Node?

In the ECG beat graph, each node is one detected heartbeat. The code first detects R-peaks in an ECG window, then extracts a small waveform segment around each R-peak.

For example, if the configuration uses 250 ms before the R-peak and 400 ms after it, each node contains a beat-centered waveform covering the P-QRS-T region around that heartbeat.

Each node therefore has two kinds of information:

- Beat morphology: the waveform segment around the R-peak.
- Timing context: RR-related features, usually describing the interval before/after the beat.

Conceptually:

$\text{node}_i = \{\text{beat waveform around } R_i, \text{RR timing features around } R_i\}$

This is better than feeding raw ECG samples directly when the task depends on heartbeat-

level structure. The model can learn whether a beat shape is sharp, noisy, wide, shifted, or repeated, while also knowing whether the rhythm around it is fast, slow, or irregular.

How Are Nodes Created?

The beat graph pipeline does roughly this:

1. Load a fixed ECG window, such as 10 or 30 seconds.
2. Normalize the signal.
3. Detect R-peaks.
4. Extract beat segments around each R-peak.
5. Compute RR features.
6. Pad variable-length beat lists so batches can be processed together.
7. Create a valid mask so padded nodes are ignored.

This matters because different windows contain different numbers of heartbeats. A 30-second window from a faster heart rate has more beats than a slower one. Padding is only a batching convenience; the valid mask tells the model which nodes are real.

How Is a Beat Encoded?

Each beat waveform is passed through a small one-dimensional convolutional network. The convolutional layers summarize the local waveform morphology into a vector. Then the model concatenates RR features to that vector and projects the result into a model dimension such as 128.

The beat encoder is doing this:

beat waveform -> convolutional feature extractor -> morphology embedding
morphology embedding + RR features -> beat node embedding

The output is a tensor shaped like:

[batch, number_of_beats, embedding_dim]

Each row is now a learned representation of one heartbeat.

What Are Edges?

There are two graph styles visible in the repository.

In the downstream supervised beat graph, the graph is built dynamically from node embeddings. The code normalizes node vectors, computes similarity between all pairs of beats in the window, then connects each beat to its top-k most similar beats. This is a k-nearest-neighbor graph in learned feature space.

That means if beat 3 looks similar to beats 9, 12, and 14, the model can pass information among them even if they are not adjacent in time.

This is useful because repeated beat morphology can be meaningful. If one beat looks suspicious but many related beats share the same pattern, the graph can stabilize the interpretation. If one beat is isolated or noisy, its context can also help.

In the pretraining model, the graph-like layer is implemented as a residual graph attention layer. Instead of explicitly using a fixed adjacency matrix, each node computes attention over other valid nodes. This lets the model learn which beats should influence each other.

So the project uses two closely related ideas:

- Explicit graph: build top-k similarity edges, then aggregate neighbor messages.
- Attention graph: let the model learn soft relationships between valid beat nodes.

Both are graph learning ideas because node representations are updated using information from other nodes.

What Is Message Passing?

Message passing is the core operation in a graph neural network. A node updates itself by combining:

- its current embedding,
- aggregated information from neighboring nodes,
- sometimes a learned gate or attention weight.

In the supervised beat graph, the GNN layer computes neighbor aggregation with the adjacency matrix. It averages neighbor embeddings, applies learned transformations, and uses a gate to

control how much neighbor information enters each node.

Intuitively:

updated beat_i = function(beat_i, average_of_related_beats)

The gate is important. It lets the model decide whether neighbor information is helpful. Some beats should be strongly influenced by neighbors. Others may need to preserve their own local evidence.

Why Not Just Use a Transformer?

A transformer can model temporal relations, but the graph adds a different bias. A pure temporal model mostly sees order. The graph can connect beats that are similar even if they are far apart within the window.

For ECG, both are useful:

- temporal order captures rhythm progression,
- graph similarity captures repeated morphology and related beat patterns.

The pretraining beat model combines both. It first uses a temporal encoder over beat embeddings, then applies graph/attention refinement. This gives the representation both sequential context and relational context.

Self-Supervised Learning for Beat Graphs

The project uses self-supervised learning so the model can learn from ECG recordings without needing labels for every window.

There are two major pretraining signals:

1. Masked node reconstruction.
2. BYOL-style consistency between two augmented views.

Masked Node Reconstruction

The model randomly masks some beat nodes. It replaces their embeddings with a learned mask

token. Then it asks the model to reconstruct the target representation for those hidden beats using the surrounding valid beats.

This is similar in spirit to masked language modeling, except the tokens are heartbeats rather than words.

The model must answer:

Given the other beats and the rhythm context, what should this missing beat representation look like?

This forces the model to learn regularities in ECG structure:

- how nearby beats usually relate,
- what normal beat morphology looks like in a window,
- how RR timing constrains expected beats,
- which patterns are stable across a recording.

BYOL-Style View Consistency

The dataset creates two augmented versions of the same ECG beat graph. Augmentations include small amplitude scaling, Gaussian noise, time masking inside beats, RR jitter, and sometimes beat dropout.

The model is trained so the two views produce similar high-level representations. This encourages robustness. The model should not change its interpretation just because the signal has small noise or mild perturbations.

In plain language:

The same ECG window should still mean the same thing after realistic noise.

Pooling: From Beat Nodes to Window Representation

Downstream tasks usually need one prediction per ECG window, not one prediction per beat. After message passing, the model pools node embeddings into a single window embedding.

The supervised beat graph uses attention pooling. Each beat receives a learned importance score. The final window representation is a weighted sum of beat embeddings.

This is helpful because not every beat contributes equally. Some beats may be noisy. Some may be more informative about state, rhythm, or quality. Attention pooling lets the model emphasize the useful beats.

What the ECG Beat Graph Is Learning

The beat graph model is learning representations at several levels:

- Beat morphology: what individual beats look like.
- Rhythm context: how beats are spaced through RR features.
- Similarity structure: which beats in a window resemble each other.
- Temporal structure: how beat patterns evolve across the window.
- Robustness: which features remain stable under noise and augmentation.
- Task-level signals: which graph patterns predict infant state, sleep, caregiver/infant class, or other labels.

The benefit is that the final representation is not just a compressed waveform. It is a learned summary of heartbeat morphology plus beat-to-beat relationships.

Part 2: IBI Graph

What Is IBI?

IBI means inter-beat interval. It is the time between consecutive heartbeats. It is closely related to RR interval in ECG analysis.

If R_i and $R_{(i+1)}$ are two consecutive R-peaks, then:

$$IBI_i = \text{time}(R_{(i+1)}) - \text{time}(R_i)$$

The IBI sequence ignores detailed ECG waveform shape and focuses on timing. This makes it useful for heart-rate variability, rhythm dynamics, state changes, sleep/arousal patterns, and autonomic regulation signals.

Why Build a Graph from IBI?

The IBI graph is a timing-focused model. Instead of asking, “what does each heartbeat waveform look like?”, it asks:

How does beat timing vary across the window?

That can be powerful because some states may be more strongly reflected in heart-rate variability than in beat morphology. For example:

- calm states may show different variability than active states,
- sleep and wake states may differ in rhythm regularity,
- stress/arousal may shift IBI dynamics,
- noisy or invalid intervals can be handled with quality features.

The IBI graph is also lighter than the beat waveform graph. It does not need convolution over full waveform segments. It uses compact hand-built features per interval.

What Is a Node in the IBI Graph?

Each node is one IBI position. The node feature vector contains several timing-derived values:

1. normalized IBI,
2. normalized delta-IBI,
3. local variability,
4. quality indicator.

The normalized IBI tells whether the interval is longer or shorter than typical for that window. Delta-IBI tells whether the rhythm is speeding up or slowing down. Local variability summarizes how much intervals fluctuate nearby. Quality indicates whether the interval is physiologically valid after cleaning.

Conceptually:

$\text{node}_i = \{\text{IBI}_i, \text{change_from_previous_IBI}, \text{local_variability}_i, \text{quality}_i\}$

This is a compact representation of rhythm dynamics.

IBI Cleaning and Feature Construction

The IBI pipeline includes cleanup because real ECG-derived intervals can contain artifacts. The code checks physiologically plausible ranges, such as minimum and maximum IBI in milliseconds. Values outside the range are treated as bad and filled using local median information when possible.

After cleaning, the model builds normalized features. Normalization is robust: it uses median and median absolute deviation rather than mean and standard deviation. That is a good choice for physiological signals because outliers and artifacts are common.

The IBI features are:

- IBI normalized by robust within-window statistics.
- Difference between consecutive IBIs, also robustly normalized.
- Rolling/local standard deviation around each interval.
- Binary or soft quality value.

This produces a sequence of feature vectors shaped like:

[number_of_intervals, feature_dim]

The collator pads these sequences and creates a valid mask, just as in the ECG beat graph.

How Are Edges Built in the IBI Graph?

The IBI graph builds edges using two ideas:

1. similarity edges,
2. temporal edges.

Similarity edges connect intervals whose learned embeddings are similar. The model first encodes IBI feature vectors into embeddings. Then it computes pairwise similarity and connects each node to its top-k most similar valid nodes.

Temporal edges connect consecutive intervals. If node i and node $i+1$ are adjacent in time, the graph connects them in both directions.

This combination is important:

- temporal edges preserve the natural order of rhythm,
- similarity edges connect repeated rhythm patterns across the window.

For example, a recurring short-long-short pattern may appear in multiple places. Similarity edges let the model compare those repeated structures. Temporal edges keep the local rhythm flow intact.

IBI Encoder and Temporal Encoder

The IBI model first maps each compact feature vector into a learned embedding using a small multilayer perceptron.

IBI features -> MLP encoder -> IBI node embedding

Then a transformer temporal encoder processes the sequence of IBI embeddings. The valid mask prevents padded nodes from influencing attention.

This temporal encoder gives each interval context from the full ordered sequence. After that, the graph is built from the contextual embeddings and the GNN layers perform message passing.

Masked Reconstruction in the IBI Graph

The IBI graph model also uses masked self-supervised learning. Some valid IBI nodes are replaced with a learned mask token. The model sees the unmasked rhythm context and tries to reconstruct target embeddings for the masked intervals.

This forces the model to learn what rhythm patterns are predictable from context.

For example, if intervals before and after a masked node suggest a stable heart rate, the model should reconstruct a representation close to that stable rhythm. If the surrounding context shows variability, the model must learn that too.

The model is not reconstructing the raw IBI scalar directly in the shown architecture. It reconstructs a hidden representation produced by the temporal encoder. That encourages the model to match meaningful contextual structure, not just numeric values.

Message Passing in the IBI Graph

After building the graph, each IBI node receives information from related intervals. The graph layer aggregates neighbor embeddings using the adjacency matrix and combines the neighbor information with the node's own embedding.

Intuitively:

updated interval_i = function(interval_i, similar_intervals, adjacent_intervals)

The updated representation can encode rhythm motifs such as:

- stable rhythm,
- gradual acceleration,
- gradual deceleration,
- high variability,
- isolated artifact,
- repeated timing pattern.

What the IBI Graph Is Learning

The IBI graph is learning rhythm and variability rather than waveform morphology.

It learns:

- heart-rate level within a window,
- beat-to-beat variability,
- local rhythm stability,
- repeated interval patterns,
- acceleration/deceleration structure,
- artifact-aware timing patterns,
- state-relevant autonomic dynamics.

This can complement the beat graph. The beat graph knows what the ECG waves look like. The IBI graph knows how timing behaves.

Beat Graph vs IBI Graph

The two graph designs answer different questions.

ECG beat graph:

- Node: full beat waveform plus RR features.
- Main signal: morphology plus timing.
- Strength: captures shape of heartbeats and relationships between beat morphologies.
- Useful when waveform quality, morphology, or repeated beat shape matters.
- Heavier model because it processes waveform segments.

IBI graph:

- Node: interval-level timing feature.
- Main signal: rhythm and variability.
- Strength: compact and focused on beat-to-beat timing dynamics.
- Useful when physiological state is reflected in heart-rate variability or rhythm.
- Lighter model because it does not process full beat waveforms.

They are complementary. A strong ECG foundation model may benefit from both:

- Beat graph: what the heartbeats look like.
- IBI graph: how the heartbeat timing changes.

What “Graph Learning” Means in This Project

Graph learning means learning representations from structured relationships between nodes. Here the nodes are ECG-derived events. The edges define which events are allowed or encouraged to share information.

The model learns by:

1. encoding each node,
2. connecting nodes through graph edges or attention,
3. passing messages between connected nodes,
4. pooling node information into a window representation,
5. training with self-supervised and supervised objectives.

The key shift is from:

Learn from a flat ECG signal.

to:

Learn from a structured heartbeat system where beats and intervals influence each other.

Why This Helps Downstream Tasks

Infant state, sleep, arousal, and caregiver/infant classification are unlikely to be determined by a single sample or single heartbeat. They are patterns across time.

Graphs help the model detect those patterns by making relationships explicit:

- repeated beat morphology,
- rhythm regularity,
- local variability,
- similar events across the window,
- important beats or intervals,
- noisy versus reliable nodes.

After graph learning, the model can produce a richer window-level embedding. A classifier can then use that embedding for labels with fewer labeled examples than would be needed to train everything from scratch.

Summary

The ECG beat graph converts a recording window into heartbeat nodes and learns how beat morphology and RR timing relate across the window. It is morphology-aware and rhythm-aware.

The IBI graph converts the same physiological process into interval nodes and learns timing dynamics. It is compact, variability-focused, and directly connected to rhythm behavior.

Together, these graph approaches are trying to build a representation of ECG that respects the natural structure of the signal: ECG is not just samples over time; it is a sequence of heartbeats with meaningful relationships.